CREATE APPLICATION THROUGH PROGRAMMING

Presented by Owen

CONTENTS

INTRODUCTION

DataMax Pte Ltd has just clinched a deal from an important client Stationery Track Pte Ltd.

The project goal is to create an application for Stationery Inventory Management system to analyse their stocks

LIBRARY PACKAGE

Build-in library packages

CSV = used for storing data in the files

OS = File handling operations

DISPLAY OUTCOME

Menu
if 1 = add item successfully
if 2 = search and edit item
if 3 = update quantity
if 4 = display inventory table
if 5 = save to CSV

If input is invalid = Display Error, retry

Do you want to continue? (y/n)
if y = display Menu
if n = display closing message

PYTHON CODE

Stationery Inventory Management System

```
import csv
import os
from item_calculator import get_total
```

Initial inventory data

```
inventory = {
  "Pen": {"quantity": 200, "price": 1.20},
  "Pencil": {"quantity": 250, "price": 0.80},
  "Eraser": {"quantity": 150, "price": 0.50},
  "Glue Stick": {"quantity": 100, "price": 1.10},
  "Writing Book": {"quantity": 300, "price": 1.50}
}
```

Menu

```
def menu():
  while True:
    clear_screen()
    print("=========================================")
    print("STATIONERY INVENTORY MANAGEMENT SYSTEM")
    print("=========================================")
    print("1 – To enter new stationery item")
    print("2 – To edit the stationery item")
    print("3 – To display the stationery item which was sold")
    print("4 – To display all the stationery items")
    print("5 – To save the list of all the stationery items in .CSV file")

    choice = input("\nEnter your choice: ")
    if choice == "1":
      add_item()
    elif choice == "2":
      edit_item()
    elif choice == "3":
      sell_item()
    elif choice == "4":
      display_items()
    elif choice == "5":
      save_to_csv()
    else:
      print("Invalid option! Try again.")
      continue

    while True:
        cont = input("\nDo you want to continue? (y/n): ").lower()
        if cont == "y":
          break  # This stops the retry loop and lets the menu loop back
        elif cont == "n":
          print("Goodbye!")
          return  # This fully stops the program
        else:
          print("Invalid input. Please enter 'y' or 'n'.")
```

Add item (option 1)

```
def add_item():
    name = input("Enter item name: ")
    quantity = get_int("Enter quantity: ")
    price = get_float("Enter price: ")
    inventory[name] = {"quantity": quantity, "price": price}
    print(f"\n{name} added successfully!")
```

Sell item (option 2)

```
def sell_item():
    name = input("Enter item name to sell: ").lower()
    match = None
    for key in inventory:
        if key.lower() == name:
            match = key
            break

    if match:
        available_quantity = inventory[match]["quantity"]
        print(f"Available quantity of {match}: {available_quantity}")
        quantity_to_sell = get_int("Enter quantity to sell: ")

        if quantity_to_sell > available_quantity:
            print("Not enough stock! Try again.")
        else:
            inventory[match]["quantity"] -= quantity_to_sell
            total_price = quantity_to_sell * inventory[match]["price"]
            print(f"Sold {quantity_to_sell} of {match} for
${total_price:.2f}")
    else:
        print("Item not found!")
```

Edit item (option 3)

```
def edit_item():
    name = input("Enter item name to edit: ").lower()
    match = None
    for key in inventory:
        if key.lower() == name:
            match = key
            break

    if match:
        quantity = get_int("Enter new quantity: ")
        price = get_float("Enter new price: ")
        inventory[match] = {"quantity": quantity, "price": price}
        print(f"{match} updated successfully!")
    else:
        print("Item not found!")
```

Display item (option 4)

```
    def display_items():
        print("\nInventory:")
        print("{:<15} {:<10} {:<10} {:<10}".format("Name", "Quantity", "Price",
"Total"))
        for item, details in inventory.items():
            total = calculate_total_price(item)
            print("{:<15} {:<10} {:<10.2f} {:<10.2f}".format(item, details["quantity"],
    details["price"], total))
    total_items = len(inventory)
    total_value = sum(calculate_total_price(item) for item in inventory)
```

Save to CSV (option 5)

```
def save_to_csv():
  with open("inventory.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Quantity", "Price", "Total"])
    for item, details in inventory.items():
      writer.writerow([item, details["quantity"], details["price"], calculate_total_price(item)])
  print("Inventory saved to inventory.csv")
```

Item_calculate

```
class Item:
  def __init__(self, name, quantity, price):
    self.name   = name
    self.quantity = quantity
    self.price   = price
```

FUNCTION

Calculate total price

```
def calculate_total_price(item):
return round(get_total(inventory[item]["quantity"],
inventory[item]["price"]), 2)
```

Get total

```
def get_total(quantity, price):
 return round(quantity * price, 2)
```

Price

```
def get_int(quantity):
  while True:
    try:
      value = int(input(quantity))
      if value < 0:
        print("Must be a positive number. Try again.")
      else:
        return value
    except ValueError:
      print("Invalid input! Please enter a whole number.")
```

Quantity

```
def get_float(price):
  while True:
    try:
      value = float(input(price))
      if value < 0:
        print("Must be a positive number. Try again.")
      else:
        return value
    except ValueError:
      print("Invalid input! Please enter a valid number.")
```

PROGRAM DEMO

FINDING AND INSIGHTS

No.	Name	Quantity	Price ($)
1	Pen	200	1.20
2	Pencil	250	0.80
3	Eraser	150	0.50
4	Glue stick	100	1.10
5	Writing	300	1.50
	And many more...		

```
— Summary ——————————————————————————————
Total inventory value    : $1075.00
Average unit price       : $1.02
Highest stock item       : Writing Book (300 units)
Lowest stock item        : Glue Stick (100 units)
Highest unit price       : Writing Book ($1.50)
Lowest unit price        : Eraser ($0.50)
Highest total value item : Writing Book ($450.00)
```

CONCLUSION

I can improve my programing 2 project by adding more categories and refinements such as :

- Sales history tracking
- Total sale summary
- Low stock alert when certain quantity is hit
- User login system to prevent unauthorised access

THANK YOU & Q&A